DAY 5 – PYTHON FUNCTIONS FOR AWS DATA ENGINEERING

Today we will learn:

1. What is a Function?
2. Why do we use Functions?
3. Creating a Function using def
4. Calling a Function
5. Function with Parameters
6. Function with Multiple Parameters
7. Arguments
8. Default Parameters
9. return Statement
10. Function Returning Multiple Values
11. Function with if-else
12. Function with Loops
13. Function with Lists
14. Function with Dictionaries
15. Function with List of Dictionaries
16. File Validation Function
17. Pipeline Status Function
18. Record Count Validation
19. S3 File Processing Example
20. Reusable Data Engineering Functions
21. Practice Programs

---

1. What Is a Function?

A **function** is a reusable block of code that performs a particular task.

For example, suppose every day we need to display:

```
print("Starting Data Pipeline")
print("Reading source data")
print("Processing data")
print("Pipeline completed")
```

Output

Starting Data Pipeline

Reading source data

Processing data

Pipeline completed

This works.

But suppose we need to execute the same code many times.

Instead of writing the same code again and again, we can put it inside a **function**.

---

2. Why Do We Use Functions?

Functions help us:

Avoid duplicate code

↓

Reuse the same logic

↓

Make code easier to understand

↓

Make code easier to maintain

↓

Break a large pipeline into smaller tasks

In Data Engineering, we can create separate functions for:

Read data

Validate data

Transform data

Load data

Check file status

Check record count

Process S3 files

For example:

S3 File

↓

read_file()

↓

validate_file()

↓

transform_data()

↓

load_data()

Each function performs one specific job.

---

3. Creating a Function Using def

In Python, we create a function using:

def

Basic syntax:

```
def function_name():
    code
```

Example:

```
def show_message():
    print("Welcome to Python")
```

Here:

def

means we are defining a function.

show_message

is the function name.

()

is where parameters can be provided.

:

indicates the start of the function block.

The code inside the function must be properly **indented**.

---

4. Calling a Function

Creating the function alone will not execute it.

Example:

```
def show_message():
    print("Welcome to Python")
```

There is no output yet.

We need to **call the function**.

```
def show_message():
    print("Welcome to Python")

show_message()
```

Output

Welcome to Python

Understand the flow:

```
def show_message():
```

↓

Function Created

show_message()

↓

Function Called

↓

Code Executes

---

5. Simple Data Engineering Function

Let's create a pipeline function.

```
def start_pipeline():
    print("Pipeline started")
    print("Reading source data")
    print("Processing data")
    print("Pipeline completed")

start_pipeline()
```

Output

Pipeline started

Reading source data

Processing data

Pipeline completed

When Python reaches:

start_pipeline()

it goes to:

```
def start_pipeline():
```

and executes all the indented code inside that function.

---

6. Calling the Same Function Multiple Times

The main advantage of a function is reusability.

```
def process_file():
    print("Processing file")

process_file()
process_file()
process_file()
```

Output

Processing file

Processing file

Processing file

We created the logic only once:

```
def process_file():
    print("Processing file")
```

But we reused it three times.

---

7. Function with Parameter

Sometimes we want to send data into a function.

For example, instead of always processing the same file, we want to provide different file names.

```
def process_file(file_name):
    print("Processing:", file_name)

process_file("customers.csv")
```

Output

Processing: customers.csv

Here:

file_name

is called a **parameter**.

And:

customers.csv

is the value we send to the function.

---

8. Parameter vs Argument

This is important.

Look at:

```
def process_file(file_name):
    print("Processing:", file_name)

process_file("customers.csv")
```

Here:

file_name

is the **parameter**.

"customers.csv"

is the **argument**.

Simple way to remember:

Parameter

↓

Variable written while creating the function

Argument

↓

Actual value passed while calling the function

---

9. Function with Different Arguments

We can call the same function with different files.

```
def process_file(file_name):
    print("Processing:", file_name)

process_file("customers.csv")
process_file("orders.csv")
process_file("products.csv")
```

Output

Processing: customers.csv

Processing: orders.csv

Processing: products.csv

The same function is reused for all three files.

---

10. Function with Multiple Parameters

A function can receive more than one parameter.

Example:

```
def file_details(file_name, record_count):
    print("File Name:", file_name)
    print("Record Count:", record_count)

file_details("customers.csv", 1000)
```

Output

File Name: customers.csv

Record Count: 1000

Here we have two parameters:

file_name

record_count

And two arguments:

"customers.csv"

1000

---

11. Data Engineering Example with Multiple Parameters

```
def pipeline_details(source, target, records):
    print("Source:", source)
    print("Target:", target)
    print("Records:", records)

pipeline_details("S3", "Redshift", 5000)
```

Output

Source: S3

Target: Redshift

Records: 5000

This function can be reused for another pipeline.

```
def pipeline_details(source, target, records):
    print("Source:", source)
    print("Target:", target)
    print("Records:", records)

pipeline_details("S3", "Redshift", 5000)
pipeline_details("RDS", "S3", 2500)
```

Output

Source: S3

Target: Redshift

Records: 5000

Source: RDS

Target: S3

Records: 2500

---

12. Default Parameter

Sometimes we want a parameter to have a default value.

Example:

```
def pipeline_status(status="SUCCESS"):
    print("Pipeline Status:", status)

pipeline_status()
```

Output

Pipeline Status: SUCCESS

We didn't provide an argument.

Python automatically used:

SUCCESS

because it is the default value.

---

13. Override the Default Value

We can also provide another value.

```
def pipeline_status(status="SUCCESS"):
    print("Pipeline Status:", status)

pipeline_status("FAILED")
```

Output

Pipeline Status: FAILED

So:

pipeline_status()

uses the default value.

But:

pipeline_status("FAILED")

uses the value we provided.

---

14. What Is return?

This is one of the most important function concepts.

```
print() displays a value.
```

return sends a value **back from the function**.

Example:

```
def add_records(a, b):
    total = a + b
    return total

result = add_records(1000, 2000)
print(result)
```

Output

3000

Let's understand this step by step.

We call:

add_records(1000, 2000)

Inside the function:

a = 1000

b = 2000

Then:

```
total = a + b
```

So:

```
total = 3000
```

Then:

return total

sends 3000 back.

It gets stored inside:

result

Finally:

```
print(result)
```

displays:

3000

---

15. print() vs return

Students should understand this difference.

Using print()

```
def add_records(a, b):
    print(a + b)

add_records(1000, 2000)
```

Output:

3000

This only displays the value.

Using return

```
def add_records(a, b):
    return a + b

total = add_records(1000, 2000)
print("Total Records:", total)
```

Output:

Total Records: 3000

With return, we can store and reuse the result later.

---

16. Reusing a Returned Value

```
def calculate_total(a, b):
    return a + b

total = calculate_total(1000, 2000)
final_total = total + 500
print("Final Total:", final_total)
```

Output

Final Total: 3500

This is why return is very useful.

---

17. Function Returning Multiple Values

A Python function can return multiple values.

```
def pipeline_result():
    status = "SUCCESS"
    records = 5000
    return status, records

pipeline_status, record_count = pipeline_result()
print("Status:", pipeline_status)
print("Records:", record_count)
```

Output

Status: SUCCESS

Records: 5000

The function returns:

SUCCESS

5000

We store them in:

pipeline_status

record_count

---

18. Function with if-else

We can use conditions inside functions.

```
def check_status(status):
    if status == "SUCCESS":
        print("Pipeline completed successfully")
    else:
        print("Pipeline failed")

check_status("SUCCESS")
```

Output

Pipeline completed successfully

Notice the indentation carefully.

def

↓

if

↓

print

else

↓

print

The if is inside the function.

The print() is inside the if.

---

19. Failed Pipeline Example

```
def check_status(status):
    if status == "SUCCESS":
        print("Pipeline completed successfully")
    else:
        print("Pipeline failed")

check_status("FAILED")
```

Output

Pipeline failed

---

20. File Validation Function

Suppose we want to process only CSV files.

```
def validate_file(file_name):
    if file_name.endswith(".csv"):
        print("Valid file:", file_name)
    else:
        print("Invalid file:", file_name)

validate_file("customers.csv")
validate_file("notes.txt")
```

Output

Valid file: customers.csv

Invalid file: notes.txt

This is a simple reusable file-validation function.

---

21. Returning Validation Result

Instead of only printing the result, we can return True or False.

```
def validate_file(file_name):
    if file_name.endswith(".csv"):
        return True
    else:
        return False

result = validate_file("customers.csv")
print(result)
```

Output

True

Now another part of the program can use this result.

---

22. Using Returned Validation Result

```
def validate_file(file_name):
    if file_name.endswith(".csv"):
        return True
    else:
        return False

file_name = "customers.csv"

if validate_file(file_name):
    print("Start processing:", file_name)
else:
    print("Skip file:", file_name)
```

Output

Start processing: customers.csv

This is closer to how reusable validation logic is used in a pipeline.

---

23. Function with a Loop

Functions can contain loops.

```
def process_files(files):
    for file in files:
        print("Processing:", file)

files = [
    "customers.csv",
    "orders.csv",
    "products.csv"
]

process_files(files)
```

Output

Processing: customers.csv

Processing: orders.csv

Processing: products.csv

Here we are passing an entire list to the function.

---

24. How the Previous Program Works

First we create the list:

```
files = [
    "customers.csv",
    "orders.csv",
    "products.csv"
]
```

Then:

process_files(files)

passes that list into the function.

Inside:

for file in files:

Python takes one file at a time.

Then:

```
print("Processing:", file)
```

prints each file.

---

25. Function with List and Condition

Now let's combine:

Function

+

List

+

Loop

+

Condition

Program:

```
def process_csv_files(files):
    for file in files:
        if file.endswith(".csv"):
            print("Processing:", file)
        else:
            print("Skipping:", file)

files = [
    "customers.csv",
    "notes.txt",
    "orders.csv",
    "readme.txt"
]

process_csv_files(files)
```

Output

Processing: customers.csv

Skipping: notes.txt

Processing: orders.csv

Skipping: readme.txt

This combines concepts from Day 2, Day 3, Day 4 and Day 5.

---

26. Function with Dictionary

Suppose we have:

```
file_info = {
    "file_name": "customers.csv",
    "status": "SUCCESS",
    "records": 1000
}
```

We can pass the dictionary to a function.

```
def show_file_details(file_info):
    print("File:", file_info["file_name"])
    print("Status:", file_info["status"])
    print("Records:", file_info["records"])

file_info = {
    "file_name": "customers.csv",
    "status": "SUCCESS",
    "records": 1000
}

show_file_details(file_info)
```

Output

File: customers.csv

Status: SUCCESS

Records: 1000

---

27. Function with List of Dictionaries

This is very useful for Data Engineering.

```
def process_files(files):
    for file in files:
        print("File:", file["name"])
        print("Status:", file["status"])

files = [
    {"name": "customers.csv", "status": "SUCCESS"},
    {"name": "orders.csv", "status": "FAILED"},
    {"name": "products.csv", "status": "SUCCESS"}
]

process_files(files)
```

Output

File: customers.csv

Status: SUCCESS

File: orders.csv

Status: FAILED

File: products.csv

Status: SUCCESS

---

28. Process Only Successful Files

Now let's make the previous program more realistic.

```
def process_success_files(files):
    for file in files:
        if file["status"] == "SUCCESS":
            print("Processing:", file["name"])
        else:
            print("Skipping failed file:", file["name"])

files = [
    {"name": "customers.csv", "status": "SUCCESS"},
    {"name": "orders.csv", "status": "FAILED"},
    {"name": "products.csv", "status": "SUCCESS"}
]

process_success_files(files)
```

Output

Processing: customers.csv

Skipping failed file: orders.csv

Processing: products.csv

---

29. Record Count Validation Function

In a Data Engineering pipeline, we may need to check whether a file contains records.

```
def validate_record_count(record_count):
    if record_count > 0:
        return True
    else:
        return False

result = validate_record_count(5000)
print(result)
```

Output

True

If:

```
result = validate_record_count(0)
```

the result will be:

False

---

30. Practical Record Validation

```
def validate_record_count(file_name, record_count):
    if record_count > 0:
        print(file_name, "- Validation Passed")
    else:
        print(file_name, "- Validation Failed")

validate_record_count("customers.csv", 5000)
validate_record_count("orders.csv", 0)
```

Output

customers.csv - Validation Passed

orders.csv - Validation Failed

---

31. Calculate Total Records Using Function

```
def calculate_total_records(record_counts):
    total = 0
    for count in record_counts:
        total += count
    return total

records = [1000, 2500, 500]
total_records = calculate_total_records(records)
print("Total Records:", total_records)
```

Output

Total Records: 4000

Let's understand the important part.

Initially:

```
total = 0
```

First iteration:

```
total = 0 + 1000
```

= 1000

Second:

```
total = 1000 + 2500
```

= 3500

Third:

```
total = 3500 + 500
```

= 4000

Then:

return total

returns 4000.

---

32. AWS S3 File Processing Example

For now, we will simulate S3 file names using a Python list.

Later, when we learn boto3, the file names can come directly from an S3 bucket.

```
def process_s3_files(files):
    for file in files:
        if file.endswith(".csv"):
            print("Processing S3 file:", file)
        else:
            print("Skipping unsupported file:", file)

s3_files = [
    "customers.csv",
    "orders.csv",
    "notes.txt",
    "products.csv"
]

process_s3_files(s3_files)
```

Output

Processing S3 file: customers.csv

Processing S3 file: orders.csv

Skipping unsupported file: notes.txt

Processing S3 file: products.csv

---

33. Reusable Data Engineering Functions

A real pipeline can be divided into smaller functions.

For example:

```
def read_data():
    print("Reading data from S3")

def validate_data():
    print("Validating data")

def transform_data():
    print("Transforming data")

def load_data():
    print("Loading data into Redshift")
```

Now call them:

read_data()validate_data()transform_data()load_data()

Output

Reading data from S3

Validating data

Transforming data

Loading data into Redshift

Instead of keeping everything in one large block, we have separated the pipeline into reusable functions.

---

34. Main Pipeline Function

We can also create one main function.

```
def read_data():
    print("Reading data from S3")

def validate_data():
    print("Validating data")

def transform_data():
    print("Transforming data")

def load_data():
    print("Loading data into Redshift")

def run_pipeline():
    print("Pipeline Started")
    read_data()
    validate_data()
    transform_data()
    load_data()
    print("Pipeline Completed")

run_pipeline()
```

Output

Pipeline Started

Reading data from S3

Validating data

Transforming data

Loading data into Redshift

Pipeline Completed

This gives us a simple pipeline flow:

run_pipeline()

↓

read_data()

↓

validate_data()

↓

transform_data()

↓

load_data()

↓

Pipeline Completed

---

35. Real Data Engineering Scenario

Now let's combine the concepts.

Requirement:

Receive multiple files

↓

Check file status

↓

Check file type

↓

Check record count

↓

Process only valid files

Program:

```
def validate_file(file):
    if file["status"] != "SUCCESS":
        return False
    if not file["name"].endswith(".csv"):
        return False
    if file["records"] <= 0:
        return False
    return True

def process_files(files):
    for file in files:
        if validate_file(file):
            print("Processing:", file["name"], "- Records:", file["records"])
        else:
            print("Skipping:", file["name"])

files = [
    {"name": "customers.csv", "status": "SUCCESS", "records": 1000},
    {"name": "orders.csv", "status": "FAILED", "records": 500},
    {"name": "notes.txt", "status": "SUCCESS", "records": 100},
    {"name": "products.csv", "status": "SUCCESS", "records": 750}
]

process_files(files)
```

Output

Processing: customers.csv - Records: 1000

Skipping: orders.csv

Skipping: notes.txt

Processing: products.csv - Records: 750

What Is Happening Here?

For every file:

validate_file(file)

checks three things.

First:

if file["status"] != "SUCCESS": return False

The file must be successful.

Second:

if not file["name"].endswith(".csv"): return False

The file must be CSV.

Third:

if file["records"] <= 0: return False

The file must contain records.

If all validations pass:

return True

Then the file is processed.

This is a good example of why functions are useful in Data Engineering.

---

36. Practice Program 1 – Simple Function

Create:

```
def welcome():
    print("Welcome to AWS Data Engineering")

welcome()
```

Expected Output

Welcome to AWS Data Engineering

---

37. Practice Program 2 – File Parameter

```
def process_file(file_name):
    print("Processing:", file_name)

process_file("sales.csv")
```

Expected Output

Processing: sales.csv

---

38. Practice Program 3 – Two Parameters

```
def show_records(file_name, records):
    print("File:", file_name)
    print("Records:", records)

show_records("customers.csv", 5000)
```

Expected Output

File: customers.csv

Records: 5000

---

39. Practice Program 4 – Return

```
def calculate_records(source_records, new_records):
    return source_records + new_records

total = calculate_records(5000, 2000)
print("Total Records:", total)
```

Expected Output

Total Records: 7000

---

40. Practice Program 5 – Function + Condition

```
def check_pipeline(status):
    if status == "SUCCESS":
        print("Pipeline Successful")
    else:
        print("Pipeline Failed")

check_pipeline("SUCCESS")
```

Expected Output

Pipeline Successful

---

41. Practice Program 6 – Function + Loop

```
def process_files(files):
    for file in files:
        print("Processing:", file)

files = [
    "customers.csv",
    "orders.csv",
    "products.csv"
]

process_files(files)
```

Expected Output

Processing: customers.csv

Processing: orders.csv

Processing: products.csv

---

42. Important Indentation to Remember

For a simple function:

```
def show_message():
    print("Hello")
```

For function + if:

```
def check_status(status):
    if status == "SUCCESS":
        print("Success")
    else:
        print("Failed")
```

For function + loop:

```
def process_files(files):
    for file in files:
        print(file)
```

For function + loop + condition:

```
def process_files(files):
    for file in files:
        if file.endswith(".csv"):
            print("Processing:", file)
```

The indentation level changes like this:

def

  for

    if

      print

This is very important when you **copy the program and run it on EC2**.

---

43. Day 5 Summary

Students should now understand:

Function

↓

def

↓

Calling a Function

↓

Parameters

↓

Arguments

↓

Multiple Parameters

↓

Default Parameters

↓

return

↓

Multiple Return Values

↓

Function + if/else

↓

Function + Loop

↓

Function + List

↓

Function + Dictionary

↓

File Validation

↓

Record Validation

↓

Reusable Data Engineering Functions

↓

Simple Pipeline Design

So our course flow is now:

Day 1: Python Fundamentals → Variables → Data Types
Day 2: Operators → Input → Type Conversion → Conditions
Day 3: Lists → Tuples → Sets → Dictionaries → List of Dictionaries
Day 4: for → range() → while → break → continue → Nested Loops
Day 5: Functions → Parameters → Arguments → return → Functions with Conditions/Loops → Reusable Data Engineering Functions